Lecture 03.18: Factorial of N (Iterative vs Recursive Formulation)

📌 Executive Summary

Factorial calculation: Mathematical definition, iterative product accumulator, recursive call stack, base cases, and recursion limits.

🔬 Theoretical Foundations & Principles

1. Mathematical Definition of Factorial

For integer $N \geq 0$, the factorial function $N!$ is defined as:

$$N! = \begin{cases} 1 & \text{if } N = 0 \\ N \cdot (N-1)! & \text{if } N \geq 1 \end{cases}$$

2. Iterative ($mathcal{O}(1)$ Aux Space) vs Recursive ($mathcal{O}(N)$ Stack Space)

- **Iterative:** Accumulates product in single loop in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space.
- **Recursive:** Allocates N `PyFrameObject` instances on call stack. If $N > 1000$, CPython raises `RecursionError` to prevent C stack overflow.

💻 Production Implementation & Code

```
def factorial_iterative(n: int) -> int:
    if n < 0:
        raise ValueError("Factorial undefined for negative integers")
    res = 1
    for i in range(2, n + 1):
        res *= i
    return res

def factorial_recursive(n: int) -> int:
```

```
    if n < 0:
        raise ValueError("Factorial undefined for negative integers")
    if n in (0, 1):
        return 1
    return n * factorial_recursive(n - 1)

print("10! (Iterative):", factorial_iterative(10))
print("10! (Recursive):", factorial_recursive(10))
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.